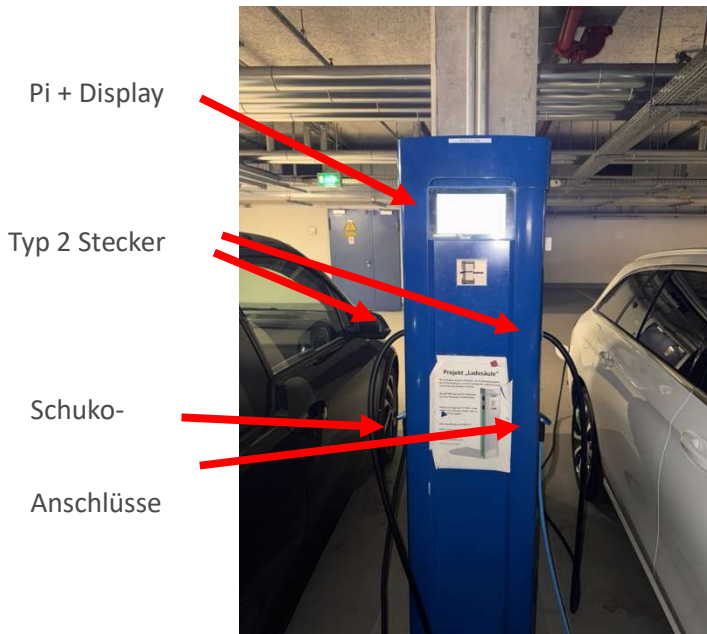


## **Projekt Lademanagement**

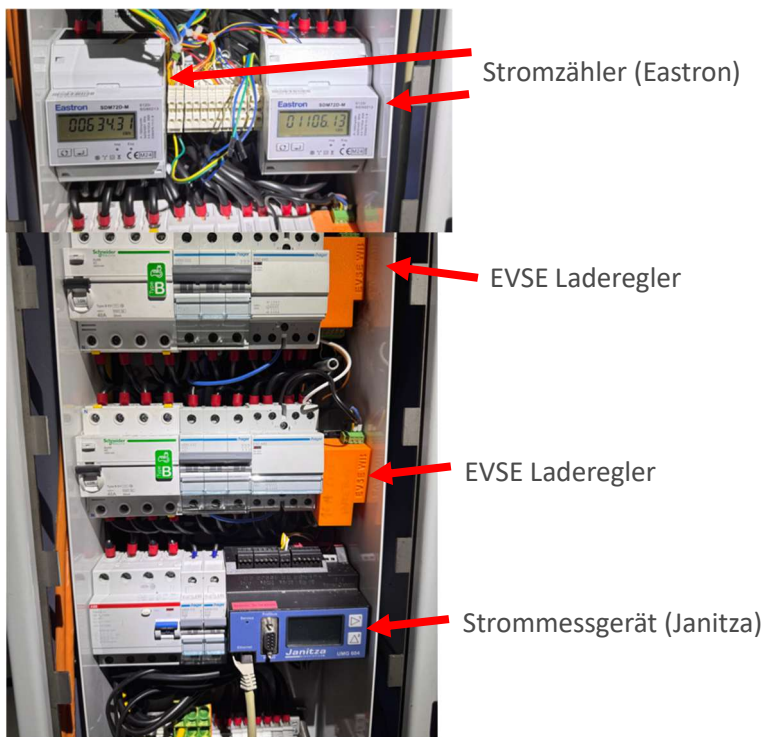
**Ziel:** Effizientes Ladesystem

Anforderungen: Keine Phase darf 32 A überschreiten

### 1. Ladesäule Aufbau (außen)



### 2. Hardwarekomponenten (innen)



#### **1. EVSE-Laderegler:**

- verbindet sich über Modbus RTU mit dem Pi und den angeschlossenen Autos
- Gibt unsere errechneten Stromlimits vor

#### **2. Strommessung (Janitza, Eastron):**

- Über Modbus TCP mit dem Pi verbunden
- Erfassen den Stromverbrauch auf den Phasen und Leiten die Daten an den Pi weiter

### 3. Softwarekomponenten:

Node-Red wurde als Grafische Entwicklungsumgebung genutzt damit der Algorithmus leichter zu bauen ist. (Lego Prinzip)

- Node-Red läuft auf dem Pi
- Empfängt Daten von EVSE und Messgeräten
- Node-Red erstellt Steuerbefehle und sendet sie an EVSE
- Node-Red erstellt das grafische Interface das auf dem Display vor Ort angezeigt wird

Für die Verwendung von Node Red auf dem PI der Ladesäule, wurde zunächst das Programm Docker installiert damit jeder Student erstmal lokal auf den Laptops den Algorithmus testen kann, bevor er auf dem Pi der Säule implementiert wird.

Docker erstellt containerisierte Umgebungen, die unabhängig vom Host-System laufen. Das bedeutet, dass Änderungen, Fehler oder Abstürze in der Node-RED-Instanz nicht das Betriebssystem deines Laptops oder des Raspberry Pi beeinträchtigen.

Öffne dein Terminal oder deine Kommandozeile und gib den folgenden Befehl ein, um das Node-RED-Image herunterzuladen: `docker pull nodered/node-red`

#### **Schritt 1: Starte einen Node-RED-Container**

Führe diesen Befehl aus, um Node-RED in einem Container zu starten:

```
docker run -d -p 1880:1880 --name mynodered nodered/node-red
```

- `-d`: Startet den Container im Hintergrund.
- `-p 1880:1880`: Leitet den Port 1880 des Containers auf deinen Host-PC weiter (Node-RED ist dann im Browser unter `http://localhost:1880` erreichbar).
- `--name mynodered`: Gibt deinem Container den Namen „mynodered“, damit du ihn leicht identifizieren kannst.
- `nodered/node-red`: Das Image, das verwendet wird.

#### **Schritt 2: Teste Node-RED**

Öffne deinen Browser und navigiere zu: `http://localhost:1880`

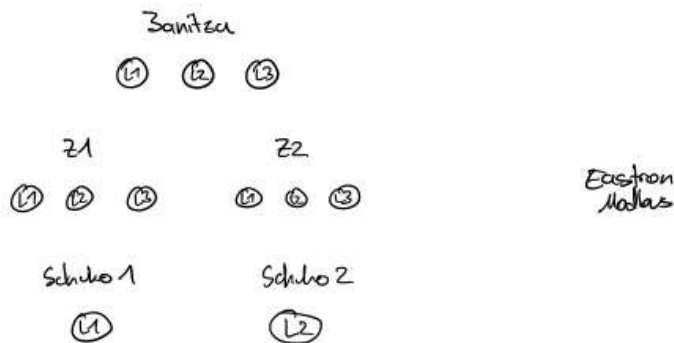
Du solltest die Node-RED-Oberfläche sehen. Du kannst jetzt Flows erstellen und testen.

Danach musst du je nach gewünschten Blöcken noch Über den Reiter Paletten ein paar Bibliotheken Installieren.

An Anfang des Projekts haben wir erstmal alle Ladefälle aufgeschrieben die Vorkommen könnten.

## Thema: Lademanagement

- Ansteuerung Laderegler über Raspi (Modbus)
- RFID Erkennung der Nutzer
- Phasen, Strom-Messung (Summen-Messung „Zanitza“, 2 3-Phasen Messgeräte „Eastron“  
Schuko-Messung aus Differenz (Zanitza - Eastron))
- Jede Phase einzeln nicht mehr als 32A



$$S1 = 3(L1) - (Z1(L1) + Z2(L1))$$

Relevanz: Effizientes Ladesystem, aktuell nicht regelbar

Probleme: Laderegler

Display kaputt

Kommunikation Raspi Modbus

RFID

Fernsteuerung über Solarlabar

Zielsetzung:

Priorisierungen:

- ① Lademanagement
  - ↳ Kommunikation Modbus
  - ↳ Messungen
  - ↳ Steuerung
- ② Fernsteuerung Solarlabar
  - ↳ Daten abnut

(Einzigste Anschaffung) ③ Display

④ RFID

⑤ Priorisierung mit vorhandenen Knöpfen

Raspi-Programme: - Node-Red (Modbus Bibliothek)  
- PHEM

(Info: bis kurz vor Ende des Projektes war uns gesagt worden, dass keine Phase 32 A überschreiten darf und dann am Schluss erst 35 A, deshalb haben wir im Folgenden immer mit 32 A als Limit gerechnet) (Zum Nachlesen auf Git nochmal hochgeladen)

### Fall 3

- 2 Autos on jew. Typ 2
- pro Auto 16 A

Fall 3.1. Auto A einphasig, Auto B dreiphasig  
→ gemeinsame Phase  
→ ~~Strom~~  $16 A$  (einphasig)

Fall 3.2. einphasig, einphasig  
→ beide gleiche Phase  
→ Wie 3.1.

Fall 3.3. einphasig, einphasig  
→ untersch. Phase  
→ Strom schrittweise erhöhen auf jew. 32 A (erstmal nur Auto A) schauen ob der Fall sehr zutrifft.

Fall 3.4. dreiphasig, dreiphasig  
→ gemeinsame Phase  
→ jeweils 16 A

3.1.1. A zieht weniger als 16 A  
→ Auto A limitieren auf aktuellen Strom verbleibt  
→ Auto B schrittweise Strom erhöhen  
Deckt ab Fall 3.1, 3.2, 3.4

### Fall 6

- 2 Autos Typ 2 + Schuko
- Laststrom =  $\frac{16A + \text{Schuko}}{2}$  (einphasig)

G.1. Typ 2 beide einphasig + Schuko

G.1.1. Alle auf einer Phase

→ beide einphasig bekommen Hälfte von Schuko übrig (32 A - Schuko) bleibt

G.1.2. beide Typ 2 einphasig, Schuko anders phasig

→ Fall 3.2. → Fall 3.1.

G.1.3. A auf Phase 1, B auf Phase 3, Schuko auf Phase 2

→ Fall 3.3.

G.1.4. A auf P1, B auf P2, Schuko auf P2 und P3

A schrittweise erhöhen auf 32 A, B limitiert auf 32 A - Schuko

G.1.5. A auf P1, B auf P2, S1-P1, S2-P2

beide limitiert auf 32 A - Schuko  
( $A = 32A - S1$ ,  $B = 32A - S2$ )

G.2. 1x einphasig, 1x dreiphasig + Schuko

G.2.1. A auf P1, B, S1 auf P1, S2 auf P2

I) B limitiert auf  $32A - S2$  1A 4A

II) A und B =  $\frac{32A - S1}{2}$  II) ↓

Von II)  $> 32A - S2 \rightarrow A = 32A - (S1 + B)$

G.2.2. A auf P1, B, S1 auf P3, S2 auf P2

B limitiert auf 32 A auf  $\text{Max}(P2, P3)$

$A + B = 16 A$

G.3. 2x dreiphasig + Schuko

G.3.1. A, B, S1 auf P1, S2 auf P2

A, B limitiert auf  $\frac{32A - \text{Max}(S1, S2)}{2}$

G.4. Sonderfälle

G.4.1. Alle selbe Phase, A zieht weniger

→ A limitiert, B erhöht bis  $32A - A - S1$

G.4.2. Fall 6.2.1. aber

G.4.2.1. A zieht weniger

solange B unter dem oberen Limit aus G.2.1 ist

→  $B = 32A - A - S1$  darf ziehen  
A limitiert

G.4.2.2. B zieht weniger

dann gilt aus G.2.1. II)

G.4.3. Fall 6.2.2. aber

G.4.3.1. A zieht weniger

A limitiert

B darf weniger als  $32A - A$  ziehen

G.4.3.2. B zieht weniger

B limitiert

A darf weniger als  $32A - B$  ziehen

G.4.4. Fall 6.2.3. aber

G.4.4.1. A zieht weniger (komplementär zu B zieht weniger)

A limitiert auf akt. Wert

B limitiert auf  $32A - \text{Max}(S1, S2) - A$

### Fall 5

- 1 Typ 2 + 1 Schuko

S.1. dreiphasig

→  $32A - \text{Schuko}$  Strom = Laststrom

S.2.1. einphasig auf Schuko Phase

→  $32A - \text{Schuko}$  Strom auf der Phase = Laststrom

#### 4. Optionaler Zubau eines RFID Card Reader

In diesem Projekt wurde versucht einen Kartenleser in die Ladesäule zu implementieren, jedoch war es noch nicht möglich die normalen Benutzerkarten die jeder besitzt auszulesen.

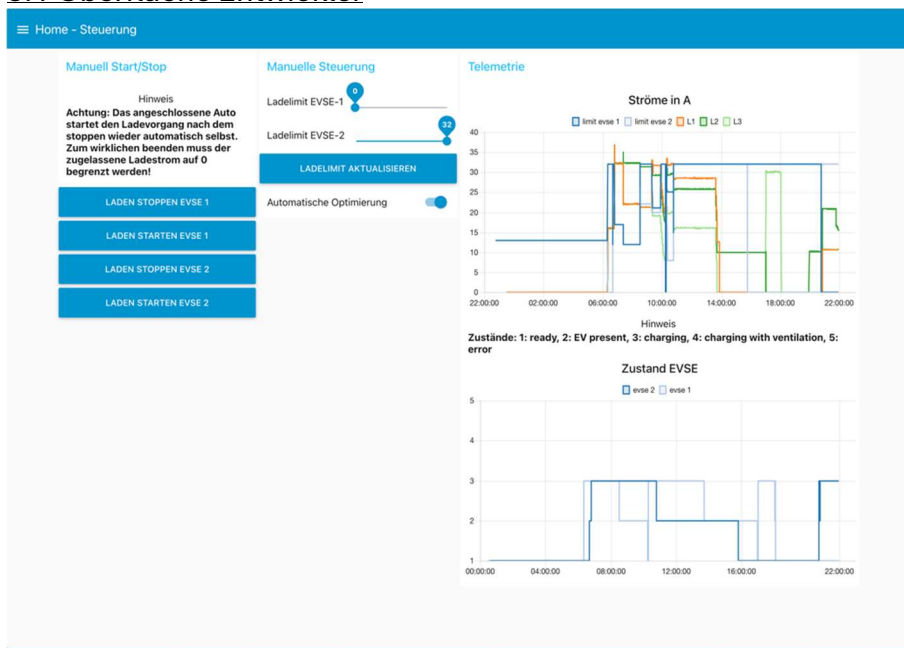
Aktuelle Recherchen haben ergeben das es sich beiden Karten um sogenannte Legic Avant Karten handeln könnte. Es sind aber auch Prime Karten im Umlauf. Für Prime Karten braucht man einen speziellen Legic Leser, weil die einen eigenen Übertragungsstandard implementiert haben. (ISO 14443) Mit normalen RFID-Readern ist es nicht möglich diese Karten auszulesen, deshalb müsste ein Legic Reader besorgt werden. (ungefähr ab 80 €)

Software zum Leser der Karten könnten sein: CTC4096-MM410, MIM256-MN/MIM1024-MN

Zusätzlich müsste noch der Node Red Algorithmus angepasst werden, es war mal gewünscht das wir Priorisiertes Laden ermöglichen sollen damit.

#### 5. Grafische Oberflächen

##### 5.1 Oberfläche Entwickler

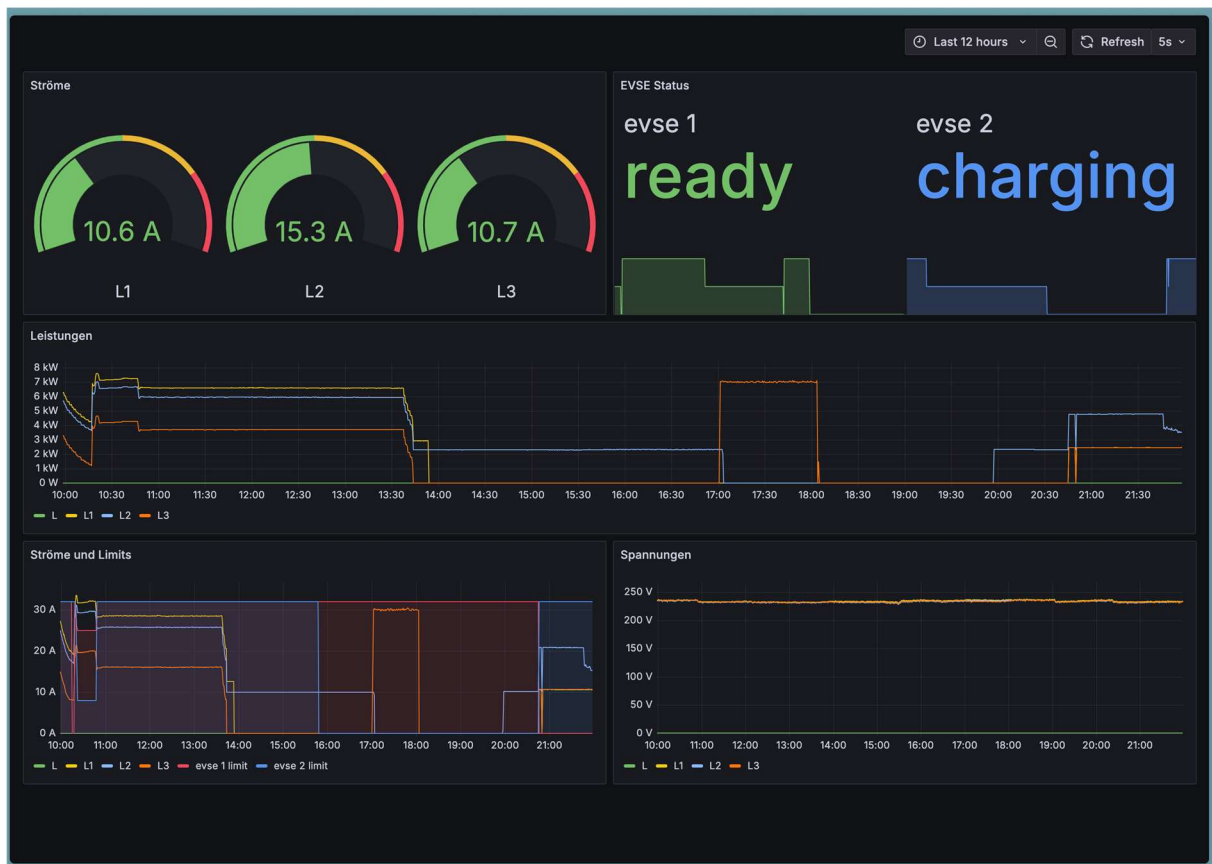


Hier in diesem Bild sieht man die Oberfläche der Steuerung, das ist nicht die Oberfläche, die jemand an der Ladesäule sieht, sondern die der Entwickler.

Damit der Ladealgorithmus auch manuell gesteuert werden kann befinden sich im Bild unter „Manuelle Steuerung“ zwei Schieber, welche den Strom der Anschlüsse limitieren.

Unter der Grafik „Zustand EVSE“ versteht man die Ladezustände der Autos, denn beim Anschluss eines Autos bekommt der EVSE über Modbus die Info in welchem Zustand sich das Auto befindet. Sollte man sich nun wundern, nach dem Anschluss seines Autos, warum es nicht lädt, könnte man über die Zustand EVSE-Grafik die Info erhalten, ob es sich evtl. in einem Fehlerzustand befindet.

## 5.2 Oberfläche Benutzer



Oben Links sind die Ströme der Phasen L1, L2, L3 zu erkennen. Rechts daneben sieht man „EVSE1 ready“ was den Zustand des TYP2 Anschluss beschreibt, denn das heißt das kein Auto angeschlossen ist. Im Gegensatz dazu ist der zweite TYP2 Anschluss belegt und lädt gerade ein Auto (EVSE2 charging).

Darunter ist die gesamte verbrauchte Leistung der Säule, aufgeteilt auf die Phasen visualisiert.

Zuletzt noch die Ströme und Spannungen der Phasen damit man sich auch die Leistung erklären kann und zusätzlich sieht, wann der Algorithmus die Ladelimits gesetzt hat bzw. wieder aufgelöst hat.